



YELIA4AP

— RECURSO EDUCATIVO —

• UNIDAD 2 •

LENGUAJE DE MODELADO UNIFICADO



Asignatura:
Programación Avanzada



Índice

2.1 Herencia.....	3
2.2 Polimorfismo	5
2.3 Clases Abstractas e Interfaces.....	8

Introducción

A medida que los sistemas informáticos aumentan en tamaño y complejidad, surge la necesidad de planificar y organizar adecuadamente su desarrollo. Antes de construir una aplicación, es importante comprender cómo estará estructurada, qué componentes la conformarán y cómo interactuarán entre sí.

En el desarrollo de software moderno, no basta únicamente con escribir código. También es necesario diseñar soluciones que permitan visualizar el funcionamiento del sistema antes de implementarlo.

En esta unidad se estudiarán conceptos fundamentales de la Programación Orientada a Objetos relacionados con la reutilización de código, la herencia, el polimorfismo, las clases abstractas y las interfaces. Estos elementos permiten construir aplicaciones más organizadas, flexibles y fáciles de mantener.

El dominio de estos conceptos constituye un paso importante para avanzar hacia el diseño de sistemas más complejos y profesionales.

2.1 Herencia

¿Qué es la herencia?

La herencia es uno de los principios fundamentales de la Programación Orientada a Objetos.

Permite que una clase pueda adquirir características y comportamientos de otra clase previamente definida.

Gracias a este mecanismo, los desarrolladores pueden reutilizar código existente y evitar repetir información innecesariamente.

La herencia establece una relación jerárquica entre clases, permitiendo compartir funcionalidades comunes.

Reutilización de código

Uno de los principales objetivos de la herencia es facilitar la reutilización del código. Cuando varias entidades poseen características similares, resulta poco eficiente crear las mismas funcionalidades una y otra vez.

La herencia permite definir elementos comunes una sola vez y reutilizarlos en diferentes partes del sistema.

Esta práctica contribuye a reducir el tiempo de desarrollo y mejora la organización del software.

Clase padre y clase hija

Dentro de la herencia participan dos tipos principales de clases.

Clase padre

Es la clase que contiene las características generales que pueden compartirse con otras clases.

- También puede recibir nombres como:
- Clase base
- Superclase

La clase padre actúa como punto de partida para la creación de nuevas clases.

Clase hija

Es la clase que hereda las características de una clase padre.

También puede recibir nombres como:

- Clase derivada
- Subclase

Además de heredar funcionalidades, la clase hija puede incorporar nuevas características propias.

Herencia simple

La herencia simple ocurre cuando una clase hija hereda directamente de una única clase padre.

Este tipo de herencia es uno de los más utilizados debido a su simplicidad y facilidad de comprensión.

Permite construir estructuras organizadas sin introducir relaciones excesivamente complejas.

Beneficios de la herencia

La herencia aporta múltiples ventajas al desarrollo de software.

Reutilización

Permite aprovechar funcionalidades ya desarrolladas.

Organización

Facilita la creación de estructuras jerárquicas.

Mantenimiento

Reduce la duplicación de código.

Escalabilidad

Facilita la incorporación de nuevas funcionalidades.

Claridad

Permite representar relaciones lógicas entre diferentes entidades.

Posibles desventajas

Aunque la herencia es muy útil, también debe utilizarse de forma adecuada.

Un uso excesivo puede generar estructuras difíciles de comprender y mantener.

Por esta razón, los desarrolladores deben analizar cuidadosamente cuándo resulta conveniente utilizar este mecanismo.

Idea clave de la sección

La herencia permite reutilizar características y comportamientos entre clases, facilitando la organización y el mantenimiento de los programas.

2.2 Polimorfismo

¿Qué es el polimorfismo?

El polimorfismo es otro de los principios fundamentales de la Programación Orientada a Objetos.

La palabra polimorfismo proviene del griego y significa "muchas formas". En programación, este concepto hace referencia a la capacidad que tiene un mismo elemento para comportarse de diferentes maneras según el contexto en el que se utilice. Gracias al polimorfismo, los programas pueden ser más flexibles y adaptarse mejor a diferentes situaciones sin necesidad de duplicar código.

Este principio permite que distintas clases compartan ciertas características comunes, pero que cada una pueda responder de manera particular cuando se ejecuta una determinada acción.

Importancia del polimorfismo

En sistemas pequeños puede parecer innecesario utilizar polimorfismo. Sin embargo, cuando una aplicación crece y maneja múltiples tipos de objetos relacionados, este principio se vuelve extremadamente útil.

El polimorfismo ayuda a:

- Reducir la duplicación de código.
- Mejorar la organización del sistema.
- Facilitar el mantenimiento.
- Incrementar la flexibilidad de las aplicaciones.
- Favorecer la reutilización de componentes.

Gracias a estas ventajas, el polimorfismo es ampliamente utilizado en aplicaciones empresariales, plataformas web, videojuegos y sistemas de gestión.

Sobrecarga de métodos

La sobrecarga de métodos consiste en utilizar un mismo nombre para diferentes métodos dentro de una clase.

Aunque los métodos comparten el mismo nombre, pueden diferenciarse por la cantidad o tipo de parámetros que reciben.

Este mecanismo permite agrupar funcionalidades relacionadas bajo un mismo nombre, facilitando la comprensión y organización del código.

La sobrecarga mejora la legibilidad de los programas y permite que una misma operación pueda ejecutarse de distintas formas según la información disponible.

Ventajas de la sobrecarga

Mayor claridad

Permite utilizar nombres coherentes para operaciones similares.

Flexibilidad

Una misma funcionalidad puede adaptarse a diferentes situaciones.

Organización

Reduce la necesidad de crear múltiples nombres para acciones relacionadas.

Sobrescritura de métodos

La sobrescritura ocurre cuando una clase hija redefine el comportamiento de un método heredado desde una clase padre.

En este caso, la clase derivada conserva la estructura general heredada, pero adapta el comportamiento a sus propias necesidades.

La sobrescritura permite personalizar funcionalidades específicas sin perder las ventajas de la herencia.

Gracias a este mecanismo, diferentes objetos pueden responder de manera distinta ante una misma acción.

Beneficios de la sobrescritura

Personalización

Cada clase puede adaptar determinados comportamientos.

Reutilización

Permite aprovechar estructuras previamente creadas.

Escalabilidad

Facilita la incorporación de nuevas funcionalidades sin modificar el diseño general.

Polimorfismo de subtipo

El polimorfismo de subtipo es una de las formas más importantes de polimorfismo.

Se produce cuando un objeto perteneciente a una clase derivada puede ser tratado como si perteneciera a su clase base.

Esto permite trabajar con diferentes tipos de objetos utilizando una misma referencia general.

Gracias a esta característica, los programas pueden diseñarse de forma más flexible y adaptable.

El polimorfismo de subtipo es ampliamente utilizado en aplicaciones que manejan múltiples tipos de entidades relacionadas entre sí.

Aplicaciones del polimorfismo en el mundo real

El polimorfismo se utiliza constantemente en el desarrollo de software moderno.

Algunos ejemplos de aplicaciones donde este principio resulta especialmente útil incluyen:

- Sistemas académicos.
- Plataformas bancarias.
- Aplicaciones móviles.
- Videojuegos.
- Sistemas empresariales.
- Plataformas de comercio electrónico.

En todos estos casos, diferentes objetos pueden compartir características comunes mientras mantienen comportamientos específicos.

Relación entre herencia y polimorfismo

La herencia y el polimorfismo suelen trabajar conjuntamente.

La herencia permite compartir características entre clases, mientras que el polimorfismo permite que dichas clases respondan de manera diferente cuando ejecutan determinadas acciones.

Gracias a la combinación de ambos principios es posible construir aplicaciones más robustas, reutilizables y fáciles de mantener.

Por esta razón, ambos conceptos son considerados pilares fundamentales de la Programación Orientada a Objetos.

Idea clave de la sección

El polimorfismo permite que una misma acción pueda ejecutarse de diferentes maneras dependiendo del objeto que la utilice, proporcionando flexibilidad y reutilización dentro del desarrollo de software.

2.3 Clases Abstractas e Interfaces

A medida que los sistemas informáticos se vuelven más complejos, los desarrolladores necesitan herramientas que les permitan diseñar aplicaciones flexibles, organizadas y fáciles de mantener.

Dentro de la Programación Orientada a Objetos existen mecanismos que ayudan a establecer estructuras comunes para diferentes clases sin necesidad de definir completamente todos sus comportamientos.

Dos de las herramientas más utilizadas para este propósito son las clases abstractas y las interfaces.

Aunque ambos conceptos tienen objetivos similares, cada uno posee características particulares que los hacen adecuados para diferentes situaciones. Comprender sus diferencias permite desarrollar software más estructurado y adaptable a futuros cambios.

¿Qué es una clase abstracta?

Una clase abstracta es una clase especial que sirve como base para otras clases. Su principal característica es que no está diseñada para crear objetos directamente. En lugar de ello, proporciona una estructura general que será utilizada por las clases que hereden de ella. Las clases abstractas permiten definir características comunes que pueden compartirse entre diferentes clases relacionadas. De esta manera se evita la duplicación de código y se mejora la organización del sistema.

Características principales de las clases abstractas

Las clases abstractas presentan diversas características importantes. Funcionan como una base común. Permiten agrupar elementos que serán compartidos por varias clases.

No se utilizan directamente

Su propósito principal es servir de modelo para otras clases.

Favorecen la reutilización

Las funcionalidades comunes pueden definirse una sola vez y utilizarse en múltiples clases.

Mejoran la organización

Permiten estructurar mejor los sistemas orientados a objetos.

Métodos abstractos

Dentro de una clase abstracta pueden existir métodos abstractos.

Un método abstracto representa una acción que debe ser implementada posteriormente por las clases derivadas.

La clase abstracta define qué acción debe existir, pero deja que cada clase hija determine cómo será realizada.

Esto proporciona flexibilidad y permite que distintas clases adapten determinados comportamientos según sus necesidades.

Ventajas de las clases abstractas

Reutilización de código

Permiten compartir funcionalidades comunes.

Organización jerárquica

Facilitan la construcción de estructuras bien definidas.

Mantenimiento simplificado

Los cambios pueden realizarse desde una ubicación central.

Mayor consistencia

Las clases derivadas siguen una estructura común.

¿Qué es una interfaz?

Una interfaz es una estructura utilizada para definir comportamientos que diferentes clases deben implementar.

A diferencia de una clase abstracta, una interfaz se centra principalmente en establecer reglas o contratos que las clases deben cumplir.

Una interfaz indica qué acciones deben existir, pero no especifica cómo deben realizarse.

Cada clase que implemente una interfaz será responsable de desarrollar sus propias soluciones.

Características principales de las interfaces

Definen comportamientos comunes

Permiten que diferentes clases compartan ciertas funcionalidades.

Promueven la flexibilidad

Las clases pueden implementar una misma interfaz de maneras distintas.

Facilitan la interoperabilidad

Permiten que diferentes componentes trabajen conjuntamente.

Favorecen diseños escalables

Ayudan a construir aplicaciones que pueden crecer sin afectar significativamente la estructura existente.

Implementación de interfaces

Cuando una clase implementa una interfaz, adquiere la responsabilidad de desarrollar todas las acciones definidas por ella.

Esto garantiza que diferentes clases puedan ofrecer funcionalidades similares, aunque internamente trabajen de manera diferente.

Gracias a este mecanismo, los desarrolladores pueden construir sistemas más consistentes y fáciles de extender.

Ventajas de las interfaces

Flexibilidad

Permiten implementar comportamientos de diversas maneras.

Independencia

Reducen el acoplamiento entre componentes.

Escalabilidad

Facilitan la incorporación de nuevas funcionalidades.

Mantenimiento

Contribuyen a desarrollar sistemas más organizados.

Diferencias entre clases abstractas e interfaces

Aunque ambos conceptos tienen objetivos similares, presentan diferencias importantes.

Clase Abstracta	Interfaz
Sirve como base para otras clases.	Define comportamientos que deben implementarse.
Puede contener funcionalidades compartidas.	Establece reglas generales.
Se utiliza para representar relaciones estrechas entre clases.	Se utiliza para compartir comportamientos comunes.
Favorece la reutilización de estructuras.	Favorece la flexibilidad del diseño.

¿Cuándo utilizar una clase abstracta?

Las clases abstractas suelen utilizarse cuando varias clases comparten características y comportamientos similares.

Resultan apropiadas cuando existe una relación fuerte entre las entidades involucradas. También son útiles cuando se desea reutilizar código común en diferentes partes de una aplicación.

¿Cuándo utilizar una interfaz?

Las interfaces suelen utilizarse cuando diferentes clases necesitan compartir ciertos comportamientos, aunque no estén relacionadas directamente.

Son especialmente útiles en sistemas grandes donde la flexibilidad y la capacidad de adaptación son aspectos importantes.

Las interfaces ayudan a construir aplicaciones más modulares y fáciles de mantener.

Aplicaciones en el desarrollo de software

Las clases abstractas y las interfaces son ampliamente utilizadas en:

- Sistemas empresariales.
- Aplicaciones web.
- Plataformas académicas.
- Aplicaciones móviles.
- Sistemas bancarios.

- Videojuegos.
- Frameworks modernos de desarrollo.

Su uso adecuado permite construir soluciones más organizadas y preparadas para futuros cambios.

Resumen General de la Unidad

En esta unidad se estudiaron conceptos fundamentales relacionados con la reutilización y organización del código dentro de la Programación Orientada a Objetos.

Se analizó la herencia como mecanismo para compartir características y comportamientos entre clases relacionadas.

Posteriormente se abordó el polimorfismo, comprendiendo cómo una misma acción puede ejecutarse de diferentes formas dependiendo del objeto que la utilice.

También se estudiaron las clases abstractas y las interfaces, herramientas que permiten establecer estructuras comunes y comportamientos compartidos dentro de aplicaciones orientadas a objetos.

Todos estos conceptos contribuyen a desarrollar software más flexible, reutilizable, organizado y fácil de mantener.

Conceptos Clave

- Reutilización de código
- Herencia
- Clase padre
- Clase hija
- Clase base
- Clase derivada
- Herencia simple
- Polimorfismo
- Sobrecarga de métodos
- Sobrescritura de métodos
- Polimorfismo de subtipo
- Clase abstracta
- Método abstracto
- Interfaz
- Implementación de interfaces
- Flexibilidad
- Escalabilidad
- Mantenimiento de software

Bibliografía

Deitel, P., & Deitel, H. Java: How to Program.

Schildt, H. Java: The Complete Reference.

Pressman, R. Ingeniería de Software: Un Enfoque Práctico.

Sommerville, I. Ingeniería de Software.

Oracle Academy. Programación Orientada a Objetos.

Oracle Java Documentation.

Microsoft Learn. Object-Oriented Programming Concepts.



GRACIAS

— POR TU ATENCIÓN —

...

